

CSTRING_REFERENCE

Core Definition

`<cstring>` is the C++ wrapper for C's `<string.h>`: null-terminated byte-string functions (`strlen`, `strcpy`, `strcmp`, ...) and raw memory functions (`memcpy`, `memset`, `memcmp`, ...). Names are available in namespace `std` (and usually the global namespace too). These functions do **no bounds checking** — the caller owns every buffer-size guarantee.

#CPP #CSTRING #C-STRINGS #MEMORY #STRCPY #MEMCPY #NULL-TERMINATED #LEGACY

COMPLETE CSTRING QUICK REFERENCE

FUNCTIONS — Target | Operation | Output

```
// =====
// LENGTH
// =====
std::strlen(s)           // const char* | Length excluding '\0' | Returns size_t; UB if not
                        // terminated
std::strnlen(s, max)     // ptr + max | Bounded length (C11 Annex K) | Returns size_t; optional,
                        // often absent

// =====
// COPYING
// =====
std::strcpy(dest, src)   // char* + src | Copy incl. '\0' | Returns dest; UB if dest too small
std::strncpy(dest, src, n) // + count | Copy at most n chars | Returns dest; NO '\0' if
                        // strlen(src) >= n

                        // Pads with '\0' if src is shorter than n
std::strcat(dest, src)   // char* + src | Append incl. '\0' | Returns dest; dest must have room
std::strncat(dest, src, n) // + count | Append at most n chars | Returns dest; ALWAYS null-
                        // terminates

                        // Writes up to n+1 bytes past dest's end
std::strdup(s)           // const char* | Duplicate via malloc | POSIX/C23; Returns char*, caller
                        // free()s
std::strndup(s, n)       // + count | Duplicate at most n | POSIX/C23; Returns char*, caller
                        // free()s

// =====
// COMPARISON
// =====
std::strcmp(a, b)        // two strings | Lexicographic compare | Returns <0, 0, >0
std::strncmp(a, b, n)    // + count | Compare first n chars | Returns <0, 0, >0
std::strcoll(a, b)       // two strings | Locale-aware compare | Returns <0, 0, >0
std::strxfrm(dest, src, n) // transform | Prepare for strcmp | Returns length needed for the
                        // transform

// =====
// SEARCHING – CHARACTERS
// =====
std::strchr(s, ch)       // string+char | Find FIRST occurrence | Returns char* or nullptr; finds
                        // '\0' too
std::strrchr(s, ch)      // string+char | Find LAST occurrence | Returns char* or nullptr
std::strspn(s, accept)   // two strings | Length of initial run | Returns size_t: chars all in
                        // `accept`
std::strcspn(s, reject)   // two strings | Length until any reject | Returns size_t
std::strpbrk(s, accept)   // two strings | First char from accept | Returns char* or nullptr

// =====
```

```
// SEARCHING – SUBSTRINGS & TOKENS
// =====
std::strstr(hay, needle)           // two strings | Find substring      | Returns char* or nullptr
std::strtok(s, delims)             // string+set | Tokenize (destructive) | Returns char* or nullptr
std::strtok(nullptr, delims)       // continue   | Next token            | MODIFIES the source; NOT thread-
safe                               //            |                       | safe
                                   //            |                       | (POSIX strtok_r / C11 strtok_s are reentrant)

// =====
// RAW MEMORY – COPY & FILL
// =====
std::memcpy(dest, src, n)          // ptrs + n   | Copy n bytes          | Returns dest; regions MUST NOT
overlap
std::memmove(dest, src, n)         // ptrs + n   | Copy n bytes safely    | Returns dest; overlap-safe
std::memset(dest, val, n)          // ptr+int+n  | Fill n bytes with val  | Returns dest; val converted to
unsigned char
std::memccpy(dest, src, c, n)      // + stop char | Copy until c or n bytes | C23/POSIX; Returns ptr past c, or
nullptr

// =====
// RAW MEMORY – COMPARE & SEARCH
// =====
std::memcmp(a, b, n)               // ptrs + n   | Compare n bytes        | Returns <0, 0, >0; no '\0'
semantics
std::memchr(ptr, ch, n)            // ptr+ch+n   | Find byte in n bytes    | Returns void* or nullptr

// =====
// ERROR MESSAGES
// =====
std::strerror(errno)               // int errno   | errno → message        | Returns char*; NOT thread-safe
                                   //            |                       | (POSIX strerror_r / C11 strerror_s are safe)

// =====
// TYPES & MACROS
// =====
std::size_t                       // Unsigned type for sizes and counts
NULL                              // C null-pointer macro – prefer nullptr in C++
```

WIDE-CHARACTER COUNTERPARTS — `<wchar>`

```
std::wcslen(ws)                   // wcslen ↔ strlen
std::wcscpy / wcsncpy             // ↔ strcpy / strncpy
std::wscat / wcsncat              // ↔ strcat / strncat
std::wcscmp / wcsncmp / wscoll    // ↔ strcmp / strncmp / strcoll
std::wcschr / wcsrchr / wcsstr    // ↔ strchr / strrchr / strstr
std::wcsnspn / wcsnspn / wcsnprk  // ↔ strspn / strcspn / strpbrk
std::wcstok(s, delims, &state)    // ↔ strtok, but reentrant (takes state ptr)
std::wmemcpy / wmemmove           // ↔ memcpy / memmove (counts are wchar_t units)
std::wmemset / wmemcmp / wmemchr  // ↔ memset / memcmp / memchr
```

COMMON PATTERNS & EXAMPLES

Safe Copying (Fixed Buffers)

```
#include <cstring>

char dest[32];
const char* src = "hello world";

// UNSAFE – no size awareness:
// std::strcpy(dest, src);

// Bounded, then force termination (strncpy does not guarantee it):
```

```
std::strncpy(dest, src, sizeof(dest) - 1);
dest[sizeof(dest) - 1] = '\0';

// Clearer alternative – measure first:
std::size_t len = std::strlen(src);
if (len < sizeof(dest)) {
    std::memcpy(dest, src, len + 1);    // +1 copies the terminator
} else {
    // truncate or fail explicitly
    std::memcpy(dest, src, sizeof(dest) - 1);
    dest[sizeof(dest) - 1] = '\0';
}
```

The strncpy Trap

```
char buf[5];
std::strncpy(buf, "hello", 5);    // Copies 'h','e','l','l','o' – NO terminator!
// std::strlen(buf);              // UB – reads past the buffer

// Correct:
std::strncpy(buf, "hello", sizeof(buf) - 1);
buf[sizeof(buf) - 1] = '\0';      // "hell"
```

Safe Concatenation

```
char buf[64] = "Log: ";

// Remaining space = capacity - current length - 1 (for '\0')
std::size_t used = std::strlen(buf);
std::size_t room = sizeof(buf) - used - 1;
std::strncat(buf, message, room);
// strncat ALWAYS null-terminates, so no manual fixup is needed.
```

Comparison and Sorting

```
#include <cstring>
#include <algorithm>

const char* names[] = {"charlie", "alpha", "bravo"};

std::sort(std::begin(names), std::end(names),
    [](const char* a, const char* b) { return std::strcmp(a, b) < 0; });

// Case-insensitive comparison (portable version – strcasecmp is POSIX):
#include <cctype>
int ciCompare(const char* a, const char* b) {
    while (*a && *b) {
        int ca = std::tolower(static_cast<unsigned char>(*a));
        int cb = std::tolower(static_cast<unsigned char>(*b));
        if (ca != cb) return ca - cb;
        ++a; ++b;
    }
    return static_cast<unsigned char>(*a) - static_cast<unsigned char>(*b);
}
```

Prefix / Suffix Checks

```
bool startsWith(const char* s, const char* prefix) {
    return std::strncmp(s, prefix, std::strlen(prefix)) == 0;
}

bool endsWith(const char* s, const char* suffix) {
    std::size_t ls = std::strlen(s), lf = std::strlen(suffix);
    return lf <= ls && std::strcmp(s + ls - lf, suffix) == 0;
}
```

Searching

```
const char* path = "/usr/local/bin/tool.exe";

const char* lastSlash = std::strrchr(path, '/');
const char* filename = lastSlash ? lastSlash + 1 : path;    // "tool.exe"

const char* dot = std::strrchr(filename, '.');
// dot → ".exe"; dot + 1 → "exe"

if (std::strstr(path, "local")) { /* substring present */ }

// First character that is one of a set:
const char* firstDigit = std::strpbrk(path, "0123456789");
```

Tokenizing with `strtok` (and why to avoid it)

```
#include <cstring>
#include <cstdio>

char line[] = "alpha,bravo,,charlie";    // MUST be modifiable – not a string literal

char* tok = std::strtok(line, ",");
while (tok) {
    std::printf("[%s]\n", tok);
    tok = std::strtok(nullptr, ",");
}
// Output: [alpha] [bravo] [charlie] – note: empty fields are SKIPPED

// Problems: destroys the input, uses hidden global state, not thread-safe,
// and cannot represent empty fields. Prefer std::string_view / std::getline.
```

Modern Tokenizing Alternative

```
#include <string>
#include <string_view>
#include <vector>

std::vector<std::string_view> split(std::string_view s, char delim) {
    std::vector<std::string_view> out;
    std::size_t start = 0;
    while (true) {
        std::size_t pos = s.find(delim, start);
        out.push_back(s.substr(start, pos - start));    // Keeps empty fields
        if (pos == std::string_view::npos) break;
        start = pos + 1;
    }
    return out;
}
```

memcpy VS memmove

```
char buf[] = "abcdefgh";

// Non-overlapping – memcpy is fine and typically fastest:
char dst[9];
std::memcpy(dst, buf, 9);

// Overlapping – memcpy is UNDEFINED BEHAVIOR; use memmove:
std::memmove(buf + 2, buf, 6); // buf → "ababcdef"
```

Zeroing and Filling

```
int arr[100];
std::memset(arr, 0, sizeof(arr)); // OK: all-zero-bits == 0 for integers

char pad[16];
std::memset(pad, ' ', sizeof(pad)); // Fill with spaces

// WRONG for non-zero values on multi-byte types:
// std::memset(arr, 1, sizeof(arr)); // Gives 0x01010101, not 1
std::fill(std::begin(arr), std::end(arr), 1); // Correct

// WRONG for non-trivial types:
// std::memset(&someStdString, 0, sizeof(std::string)); // Corrupts the object
```

Binary Comparison

```
struct Key { int a; int b; };

// memcmp compares padding bytes too – only valid if the type has no padding
// and no floating point (-0.0 vs 0.0, NaN) or pointer members.
bool sameBytes(const Key& x, const Key& y) {
    return std::memcmp(&x, &y, sizeof(Key)) == 0;
}
// Safer: write operator== field by field, or use = default (C++20).
```

Type Punning the Legal Way

```
#include <cstring>
#include <cstdint>

float bitsToFloat(std::uint32_t bits) {
    float f;
    std::memcpy(&f, &bits, sizeof(f)); // Well-defined; compilers optimize to a register move
    return f;
}
// C++20: std::bit_cast<float>(bits) – same result, constexpr, no memcpy needed.
```

Bridging to `std::string`

```
#include <cstring>
#include <string>

const char* c = "hello";
std::string s = c;           // Implicit conversion
std::string s2(c, std::strlen(c)); // Explicit length

const char* back = s.c_str(); // Null-terminated, valid until s is modified
const char* raw  = s.data();  // C++11+: also null-terminated

// Copying a std::string into a fixed C buffer:
char buf[32];
std::size_t n = s.copy(buf, sizeof(buf) - 1); // copy() does NOT null-terminate
buf[n] = '\0';
```

Error Messages from `errno`

```
#include <cstring>
#include <cerrno>
#include <cstdio>

std::FILE* f = std::fopen("missing.txt", "r");
if (!f) {
    std::fprintf(stderr, "open failed: %s\n", std::strerror(errno));
}
// Modern C++: std::error_code / std::system_error carry the same information safely.
```

IMPORTANT CONCEPTS

No Bounds Checking, Anywhere

Every function here trusts the caller. Buffer overflows from `strcpy` / `strcat` / `sprintf` are the classic C security bug class. The size guarantee must come from your code, not the library.

Null Termination Is the Contract

`strlen`, `strcpy`, `strcmp` and friends all walk memory until they hit `'\0'`. A buffer that is not terminated makes every one of them undefined behavior. The `mem*` family, by contrast, takes an explicit byte count and never looks for a terminator.

Sizing: `sizeof` VS `strlen`

```
char arr[32] = "hi";
const char* ptr = arr;

sizeof(arr);           // 32 - the array's storage
std::strlen(arr);      // 2 - characters before '\0'
sizeof(ptr);           // 8 - the POINTER's size, not the string's (classic bug)
```

An array decays to a pointer when passed to a function, so `sizeof` inside the callee is useless. Pass the size explicitly, or use `std::string` / `std::span`.

strncpy Is Not "Safe strncpy"

It was designed for fixed-width record fields, not for safety. It may leave the destination unterminated, and it pads the entire remaining buffer with `'\0'` when the source is short (a performance cost on large buffers). Always terminate manually.

strtok Modifies Its Input

It writes `'\0'` over each delimiter and keeps internal static state between calls. That means: no string literals, no concurrent use, no nested tokenizing loops. Use `strtok_r` (POSIX) / `strtok_s` (C11) if you must, or `std::string_view` if you can.

memcpy Regions Must Not Overlap

Overlap is undefined behavior even though it often "works". `memmove` handles overlap correctly and is barely slower on modern implementations.

memset Fills Bytes, Not Values

`memset(p, v, n)` writes the low byte of `v` into each of `n` bytes. Correct for zeroing and for `char` fills; wrong for setting an `int` array to 1. And it must never be applied to non-trivially-copyable types — it will destroy vtable pointers and internal invariants.

Annex K (`strcpy_s`, `strcat_s`, ...) Is Optional

The `_s` bounds-checked variants are an optional C11 annex. MSVC implements them; glibc and libc++ largely do not. Do not assume portability.

The C++ Answer Is Usually a Different Type

For nearly every task in this header, `std::string` (owning, growable) or `std::string_view` (non-owning, cheap) is safer and no slower. Reach for `<cstring>` when interfacing with C APIs, doing genuinely raw byte work, or optimizing a measured hot path.

SAFETY CHEAT SHEET

Function	Risk	Safer choice
<code>strcpy</code>	Overflow, no size limit	<code>std::string</code> ; or <code>memcpy</code> with a checked length
<code>strcat</code>	Overflow, 0(n) rescan each call	<code>std::string</code> <code>operator+=</code> ; or track the length
<code>strncpy</code>	May not null-terminate; pads	<code>memcpy</code> + explicit <code>'\0'</code> ; or <code>std::string</code>
<code>strlen</code>	UB on unterminated buffers	<code>std::string::size()</code> ; <code>string_view::size()</code>
<code>strtok</code>	Destructive, global state	<code>string_view</code> split; <code>std::getline</code> with a delimiter
<code>sprintf</code>	Overflow (in <code><stdio></code>)	<code>snprintf</code> ; <code>std::format</code> (C++20)
<code>gets</code>	Removed in C++14 — never usable	<code>std::getline</code>
<code>memcpy</code>	UB on overlap	<code>memmove</code> when regions may overlap
<code>memset</code>	Byte-fill only; UB on class types	<code>std::fill</code> ; <code>value-initialization {}</code>
<code>strerror</code>	Not thread-safe	<code>strerror_r</code> / <code>strerror_s</code> ; <code>std::system_error</code>

BEST PRACTICES

1. Prefer `std::string` and `std::string_view` — reach for `<cstring>` only at C boundaries
2. Never call `strcpy` / `strcat` into a buffer you did not size-check
3. Terminate manually after `strncpy` — it does not promise a `'\0'`

4. Pass buffer sizes alongside pointers — `sizeof` is lost across a function call
5. Use `memmove` whenever overlap is possible
6. Use `memset` only for zeroing and byte fills on trivially-copyable types
7. Avoid `strtok`; if forced, use `strtok_r` / `strtok_s`
8. Use `memcpy` (or C++20 `std::bit_cast`) for type punning — never a reinterpret-cast dereference
9. Compare with `strcmp`, never `==`, when handling `const char*`
10. Watch for signed `char` — pass unsigned `char` to `<cctype>` functions
11. Prefer `std::string::c_str()` over hand-managed buffers when calling C APIs
12. Enable `-Wall -Wextra -D_FORTIFY_SOURCE=2` and run ASan/UBSan on code that uses these functions

RELATED HEADERS

```
#include <cstring>      // strlen, strcpy, memcpy, memset ...
#include <cwchar>       // wide-character equivalents
#include <cctype>       // isalpha, tolower ... (pass unsigned char!)
#include <cwctype>      // wide character classification
#include <string>       // std::string – the C++ answer
#include <string_view>  // C++17 non-owning string view
#include <cstdlib>      // atoi, strtol, malloc/free, qsort/bsearch
#include <cstdio>       // snprintf, fopen, FILE*
#include <cerrno>       // errno, used with strerror
#include <bit>         // C++20: std::bit_cast (safer type punning)
#include <span>         // C++20: pointer+size as one parameter
#include <algorithm>    // std::copy, std::fill, std::equal – type-aware replacements
```

EXTERNAL RESOURCES

- Null-terminated byte strings: <https://en.cppreference.com/w/cpp/string/byte>
- `<cstring>`: <https://en.cppreference.com/w/cpp/header/cstring>
- C++ Core Guidelines, SL.str: <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#SS-string>

Standard: C++11+ (C99/C11 library base) | POSIX and Annex K extensions noted where relevant **Last Updated:** September 2026